	Carátula para entrega de prácticas
Facultad de Ingeniería	Laboratorios de docencia

Laboratorio de Computación Salas A y B

Profesor(a): Rene Adrián Davila Pérez

Asignatura: Programación Orientada a Objetos

Grupo: 7

No de Práctica(s): 8

Integrante(s): 323001315, 426057857, 323008754, 426058665, 323067267

No. de lista o brigada: 5

Semestre: 2027-1

Fecha de entrega: 23 de Octubre

Observaciones:

CALIFICACIÓN: _____

Índice

1. Introducción **2**
1.1. Planteamiento del problema 2
1.2. Motivación 2
1.3. Objetivos 2

2. Marco Teórico **3**
2.1. Polimorfismo 3
2.2. Clases abstractas 3
2.3. Interfaces 3
2.4. Clase abstracta frente a interfaz 3

3. Desarrollo **4**

4. Resultados **6**
4.1. Problema 1-como elaborar la GUI para hacer login de Usuario y Contraseña. 6
4.2. Problema 2-Huella de Carbono con interfaz. 7

5. Conclusiones **8**

1. Introducción

1.1. Planteamiento del problema

La práctica consta de dos ejercicios:

- **Ejercicio 1.** Consultar con una herramienta de Inteligencia Artificial Generativa (IA Gen) cómo elaborar una interfaz gráfica de usuario (GUI) para el inicio de sesión, es decir, una ventana que solicite un usuario y una contraseña. El problema consiste en identificar qué componentes y qué mecanismos del lenguaje se necesitan para construir dicha ventana y reaccionar a las acciones de la persona usuaria.
- **Ejercicio 2.** Modelar la huella de carbono de tres objetos que no tienen relación entre sí: un edificio, un auto y una bicicleta. Se deben crear tres clases pequeñas, sin relación de herencia, con atributos y comportamientos propios. Además, se debe escribir una interfaz llamada `HuellaCarbono` con el método `getHuellaCarbono()`, que cada clase implementa para calcular la huella que le corresponde. En la clase `Main` se crea un objeto de cada clase, se almacenan sus referencias en un `ArrayList<HuellaCarbono>` y se recorre la lista invocando de forma polimórfica el método `getHuellaCarbono()`, mostrando para cada objeto información que lo identifique y su huella de carbono.

1.2. Motivación

Casi todo programa que interactúa con personas necesita una interfaz gráfica, y la pantalla de inicio de sesión es uno de los puntos de entrada más comunes. Aprender a construirla permite pasar de programas que se ejecutan en consola a aplicaciones más cercanas a las que se usan a diario. Además, consultar una IA Gen para resolver este ejercicio obliga a revisar con criterio lo que sugiere, en lugar de aceptarlo sin comprobarlo.

Por otra parte, objetos tan distintos como un edificio, un auto y una bicicleta comparten una característica: todos generan emisiones. Dar solución al segundo ejercicio permite tratar a todos de la misma forma sin forzar una herencia artificial entre ellos. Con una interfaz, el código que recorre la lista no necesita saber de qué tipo es cada objeto, y se pueden agregar nuevas clases (por ejemplo, un avión) sin modificar el código existente. Esto muestra la utilidad práctica del polimorfismo.

1.3. Objetivos

Objetivo general. Aplicar el polimorfismo mediante interfaces en Java para resolver un problema de cálculo de huella de carbono, y conocer los elementos básicos para construir una interfaz gráfica de inicio de sesión.

Objetivos específicos.

- Definir una interfaz y lograr que clases sin relación de herencia la implementen.

- Sobrescribir un mismo método en cada clase para que cada objeto calcule su resultado de forma distinta.
- Almacenar referencias de distintos objetos en un `ArrayList` del tipo de la interfaz e invocar el método de manera polimórfica.
- Organizar el código en paquetes, compilarlo y documentarlo con Javadoc.
- Consultar con IA Gen la elaboración de una GUI de usuario y contraseña, e identificar los componentes y eventos que intervienen.

2. Marco Teórico

2.1. Polimorfismo

El polimorfismo es la capacidad de que una misma instrucción produzca comportamientos distintos según el objeto que la recibe. En Java, una referencia de un tipo general (una superclase, una clase abstracta o una interfaz) puede apuntar a objetos de clases diferentes. Al invocar un método mediante esa referencia, Java ejecuta en tiempo de ejecución la versión que corresponde al tipo real del objeto; a esto se le llama enlace dinámico. El polimorfismo se apoya en la sobrescritura de métodos [1].

2.2. Clases abstractas

Una clase abstracta se declara con la palabra reservada `abstract` y no se puede instanciar: existe para ser extendida por otras clases. Puede tener atributos, constructores y métodos con código, y también métodos abstractos, que solo declaran la firma y obligan a las subclases a sobrescribirlos. Se utiliza cuando varias clases relacionadas por herencia comparten estado y comportamiento [3].

2.3. Interfaces

Una interfaz define un contrato: un conjunto de métodos que las clases se comprometen a implementar, sin indicar cómo. No guarda estado de instancia (solo puede declarar constantes) y una clase puede implementar varias interfaces con la palabra reservada `implements`. Su principal utilidad es lograr polimorfismo entre clases que no están relacionadas por herencia [4].

public interface HuellaCarbono double getHuellaCarbono();

2.4. Clase abstracta frente a interfaz

En esta práctica se eligió una interfaz porque `Edificio`, `Auto` y `Bicicleta` no están relacionadas por herencia ni comparten atributos o código; lo único que tienen en común es que todas pueden calcular una huella de carbono.

Tabla 1: Comparación entre clase abstracta e interfaz.

	Clase abstracta	**Interfaz**
Se usa con	`extends`	`implements`
Atributos de instancia	Sí	No (solo constantes)
Métodos	Abstractos y con código	Abstractos (y `default` desde Java 8)
Cuántas puede tener una clase	Una sola superclase	Varias interfaces
Cuándo conviene	Clases relacionadas que comparten estado y código	Clases sin relación que comparten un comportamiento

3. Desarrollo

Ejercicio 1

1. **Formulación de la pregunta.** Se redacta la consulta para que cubra los puntos solicitados.

2. **Consulta a la IA.** Se envía la pregunta a la IA generativa y se lee la respuesta completa.

3. **Análisis de la información.** Se revisan los componentes, la estructura y los pasos que la IA propone para construir la ventana de login.

4. **Identificación del proceso de validación.** Se identifica cómo la IA recomienda comprobar el usuario y la contraseña, y las razones que da.

5. **Verificación de la información.** Se contrasta lo obtenido.

6. **Registro de la consulta.** Se anotan en el reporte la pregunta formulada, la IA utilizada y un resumen de la respuesta.

7. **Conclusión.** Se resumen los puntos principales y se indica el procedimiento que se seguirá para elaborar la GUI.

Ejercicio 2

1. **Definición de las clases.** Se identifican tres clases pequeñas que no se relacionan por herencia: `Edificio`, `Auto` y `Bicicleta`.

2. **Atributos y comportamientos propios.** A cada clase se le dan atributos y métodos exclusivos, que no comparte con las otras. Por ejemplo, un edificio tiene un consumo de energía, un auto tiene un tipo de combustible y una bicicleta tiene un tipo de uso.

3. **Creación de la interfaz HuellaCarbono.** Se crea `HuellaCarbono.java` con el método `getHuellaCarbono()`, que es el que todas las clases deben implementar.

4. **Investigación del cálculo.** Se consulta a la IA cómo calcular la huella de carbono que corresponde a cada clase, y se anotan las fórmulas obtenidas.

5. **Implementación de la interfaz.** Cada clase usa `implements HuellaCarbono` y define `getHuellaCarbono()` con el cálculo que le corresponde.

6. **Programa principal.** En `Main` se crea un objeto de cada clase y se guardan las referencias en un `ArrayList<HuellaCarbono>`.

7. **Recorrido polimórfico.** Se recorre el `ArrayList` y se invoca `getHuellaCarbono()` en cada objeto. Para cada uno se muestra información que lo identifique y su huella de carbono.

8. **Verificación de los nombres de las clases.** Se comprueba que cada clase `public` tenga el mismo nombre que su archivo.

9. **Comprobación de resultados.** Se comparan los valores mostrados con los cálculos hechos a mano.

10. **Cierre del trabajo.** Se confirma que las tres clases, la interfaz y el `Main` están completos.

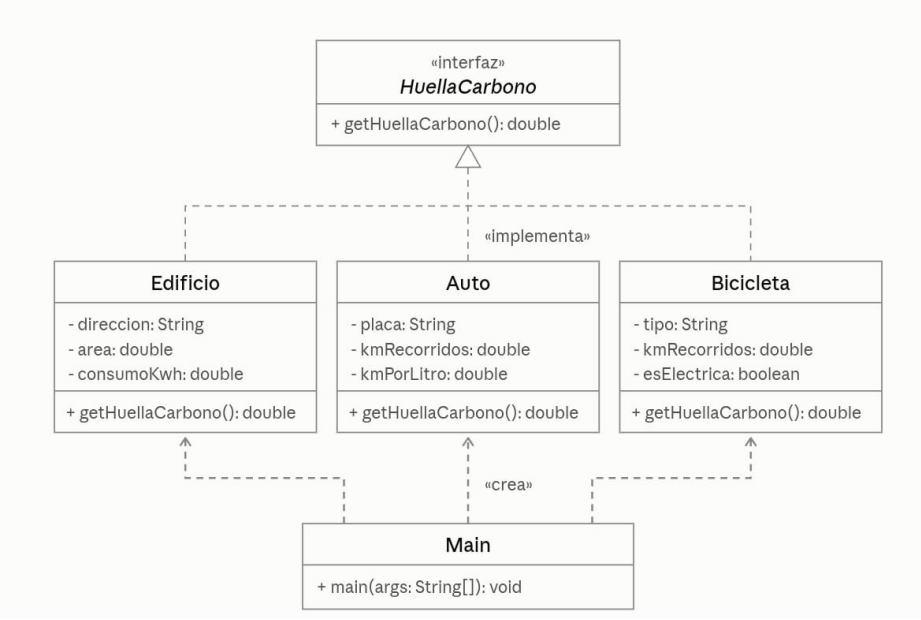


Figura 1: Diagrama de clases ejercicio 2

4. Resultados

4.1. Problema 1-como elaborar la GUI para hacer login de Usuario y Contraseña.

Para elaborar una interfaz gráfica de usuario (GUI) de login, el enfoque estándar en programación orientada a objetos utiliza **Java con la biblioteca Swing**. Esta biblioteca proporciona los componentes necesarios para ventanas, campos de texto y botones sin necesidad de instalar dependencias externas.

Aquí la estructura completa para construir una ventana de inicio de sesión funcional, cuyo diseño se fundamenta en la arquitectura orientada a eventos y la diferenciación de componentes por seguridad:

- **1. Componentes de Entrada Diferenciados:** La creación de la GUI comienza definiendo los campos donde el usuario interactuará. Es vital distinguir entre datos públicos y sensibles. El nombre de usuario se captura en un campo de texto estándar (`JTextField`), mientras que la contraseña requiere un componente especializado (`JPasswordField`) que enmascare los caracteres visualmente en la pantalla para proteger la información de miradas indiscretas.

```
JTextField txtUsuario = new JTextField();
JPasswordField txtPassword = new JPasswordField(); // Enmascara la entrada visualmente
```

Figura 2: parte fundmantela del código.

- **2. El Disparador (Event Listener):** Las interfaces gráficas no operan de forma lineal, sino mediante eventos. El programa construye la ventana y permanece inactivo a la espera de que el usuario interactúe con el botón de ingreso. A este botón (`JButton`) se le acopla un escuchador (`ActionListener`) que actúa como el gatillo para disparar la lógica de validación únicamente al detectar el clic.

```
botonIngresar.addActionListener(evento -> {
    // La lógica de validación va aquí adentro
});
```

Figura 3: parte fundmantela del código.

- **3. Extracción Segura de Credenciales:** Una vez que el evento es disparado, se procede a capturar la información. Por cuestiones críticas de seguridad en el manejo de memoria, el usuario se extrae como una cadena de texto (`String`), pero la contraseña se extrae como un arreglo de caracteres (`char[]`). A diferencia de los Strings que son inmutables y permanecen en la memoria, un arreglo de caracteres puede limpiarse o sobrescribirse de la memoria RAM inmediatamente después de su uso.

```
String usuario = txtUsuario.getText();
char[] password = txtPassword.getPassword(); // Se usa char[], no getText() por seguridad en memoria
```

Figura 4: parte fundmantela del código.

- **4. Lógica de Validación y Retroalimentación:** En esta etapa final, que ocurre dentro del evento del botón, se comparan las credenciales capturadas contra las autorizadas en el sistema. La interfaz debe proporcionar una respuesta visual al usuario mediante un `JOptionPane`. En caso de éxito, se da paso al sistema; en caso de falla, se debe lanzar un error genérico (sin especificar si falló el usuario o la contraseña) para no dar pistas a posibles atacantes.

```
// Comparación lógica
if (usuario.equals("admin") && Arrays.equals(password, "1234".toCharArray())) {
    // Lanzar diálogo de éxito y abrir nueva ventana
} else {
    // Lanzar diálogo de error de autenticación
}
```

Figura 5: parte fundmantela del código.

- **Resumen:** Crear la GUI de un login consiste esencialmente en proteger la entrada de datos con componentes especializados, implementar un paradigma orientado a eventos para capturar el intento de acceso, y aplicar buenas prácticas de manejo de memoria al extraer y validar la contraseña.

4.2. Problema 2-Huella de Carbono con interfaz.

El programa instancia tres objetos de clases distintas que no comparten relación de herencia entre sí (`Edificio`, `Auto` y `Bicicleta`), cada uno inicializado con atributos exclusivos. Estas instancias se agrupan en una colección `ArrayList` de tipo `HuellaCarbono` (la interfaz que todas implementan).

Mediante un ciclo y haciendo uso del polimorfismo, el programa recorre la lista e invoca el método `getHuellaCarbono()` de cada elemento en tiempo de ejecución. Como resultado, se imprime en consola la información identificativa de cada objeto mediante su método `toString()`, el cálculo de su respectiva huella de carbono utilizando sus factores de emisión particulares, y finalmente, la sumatoria total de las emisiones de los tres elementos.

```
PS C:\Users\acer\Downloads\Practica-8-Equipo-5-main\Practica-8-Equipo-5-main\mx\unam\fi\die\poo\g7\src> java mx.unam.fi.die.poo.g7.pra
ctica08.Main
Huella de carbono por objeto
Edificio [direccion=Av. Universidad 3000, area=1200.0 m2, consumo=8500.0 kWh, gas=300.0 m3]
  Huella de carbono: 4323.00 kg CO2

Auto [placa=ABC-123, km=15000.0, rendimiento=12.0 km/L]
  Huella de carbono: 2887.50 kg CO2

Bicicleta [tipo=Urbana, km=3000.0, electrica=false]
  Huella de carbono: 0.00 kg CO2

Huella total: 7210.50 kg CO2
PS C:\Users\acer\Downloads\Practica-8-Equipo-5-main\Practica-8-Equipo-5-main\mx\unam\fi\die\poo\g7\src>
```

Figura 6: Problema 2-Huella de Carbono con interfaz.

5. Conclusiones

El desarrollo de esta práctica evidenció la importancia de aplicar correctamente los conceptos teóricos orientados a objetos y de eventos para la resolución de problemas de software:

- **Seguridad y control de eventos en GUI:** La construcción de la interfaz gráfica comprobó que el software interactivo requiere rigor en el manejo de memoria. Extraer contraseñas mediante arreglos (`char[]`) en lugar de cadenas inmutables (`String`) mitiga vulnerabilidades, mientras que el uso de *listeners* valida la necesidad teórica de desacoplar la interfaz visual de la lógica de validación.
- **Interfaces frente a herencia:** El cálculo de emisiones demostró la utilidad de las interfaces para unificar entidades sin relación taxonómica (`Edificio`, `Auto` y `Bicicleta`). La interfaz `HuellaCarbono` estableció un contrato estandarizado efectivo, evitando forzar una jerarquía de clases artificial.
- **Polimorfismo y enlace dinámico:** Al agrupar estas referencias dispares en un solo `ArrayList` y procesarlas, se confirmó el poder del polimorfismo. Los resultados en consola ratifican que el sistema logra delegar el comportamiento específico a cada instancia en tiempo de ejecución, creando una arquitectura escalable.

Referencias

[1] J. A. Solano Gálvez, M. A. Nakayama C. y A. Velázquez Mena, *Manual de prácticas del Laboratorio de Programación orientada a objetos*, Versión 01. Ciudad de México, México: Facultad de Ingeniería, UNAM, 2017, pp. 112-126.

[2] P. Deitel y H. Deitel, *Cómo programar en Java*, 7.ª ed. Naucalpan de Juárez, México: Pearson Educación, 2008.

[3] D. J. Barnes y M. Kölling, *Programación Orientada a Objetos con Java*, 3.ª ed. Madrid, España: Pearson Educación, 2007.

[4] Oracle, *The Java Tutorials: Interfaces and Inheritance*. [En línea]. Disponible en: `https://docs.oracle.com/javase/tutorial/java/IandI/index.html`